第十一讲教学设计：Filter 过滤器与请求处理链 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: Filter过滤器与请求处理链
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet/JSP、会话管理(HttpSession)与基础MVC模式的学生

一、教学目标

知识目标

1. 理解 `Filter` 在Servlet规范中的定位与典型应用场景（统一编码Encoding、权限校验Authentication、日志记录Logging、性能统计Performance等）。
2. 掌握 `Filter` 生命周期（Lifecycle）与核心方法： `init` 、 `doFilter` 、 `destroy` 。
3. 掌握过滤器链（Filter Chain）的工作机制与 `chain.doFilter()` 的前后处理时机。
4. 掌握两种配置方式：注解（Annotation） `@WebFilter` 与 `web.xml` 中的 `<filter>` / `<filter-mapping>` ，理解拦截URL模式（URL Pattern）与调用顺序。
5. 了解 `DispatcherType` （ `REQUEST` 、 `FORWARD` 、 `INCLUDE` 、 `ERROR` ）对过滤器生效时机的影响。

能力目标

1. 能独立编写并配置“统一编码过滤器”“认证过滤器”“日志/性能过滤器”。
2. 能根据需求合理安排多个过滤器的执行顺序，实现“前置处理-业务处理-后置统计”的完整链路。
3. 能利用白名单与URL模式，避免对静态资源的无效拦截。

素质目标

1. 强化工程化与规范化意识：将共性问题抽离到“横切关注点”统一处理。
2. 建立安全思维：对关键资源进行访问控制与最小化授权。
3. **[思政融合]** 借“守门员/安检口”的比喻强调规则与秩序的重要性：人人过闸，公平透明，既保障效率也守住底线。

二、教学重点与难点

教学重点

- 过滤器链调用过程与 `chain.doFilter()` 的“前后钩子”语义。
- 多个过滤器的生效顺序与URL匹配策略（注解与 `web.xml` 配置差异）。
- 典型场景：统一编码、登录态校验、请求日志与性能统计、静态资源放行。

教学难点

- 正确认知执行顺序：外层过滤器先进入、后退出（洋葱模型）。
- `DispatcherType` 带来的调用时机差异（如转发 `FORWARD` 场景）。
- 线程安全与无状态：过滤器是单实例多线程对象，避免共享可变成员。

三、教学内容与时间安排

第一部分：问题引入——为何需要“统一入口”的处理？（4分钟）

内容要点

1. 展示多个Servlet中重复的编码设置、权限校验代码，指出维护成本高、易遗漏。
2. 提问：是否能在“进入Servlet之前”集中完成这些横切逻辑？
3. 引出Filter：容器级“前置/后置”处理器，可形成可配置的处理链。

教学方法

- 问题驱动 + 反例对比（多处重复代码 vs. 单点治理）。

第二部分：新知讲解——Filter原理与配置 (9分钟)

内容要点

1. 生命周期与核心方法：`init`（一次性初始化）→ `doFilter`（每次请求）→ `destroy`（销毁）。
2. 编码过滤器（统一UTF-8）：

```
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import java.io.IOException;

@WebFilter(filterName = "EncodingFilter", urlPatterns = "/*",
           dispatcherTypes = { DispatcherType.REQUEST, DispatcherType.FORWARD })
public class EncodingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        req.setCharacterEncoding("UTF-8");
        res.setContentType("text/html;charset=UTF-8");
        chain.doFilter(req, res); // 放行到下一个过滤器或目标资源
    }
}
```

3. 认证过滤器（保护 `/admin/*`）：

```
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebFilter(filterName = "AuthFilter", urlPatterns = { "/admin/*" })
public class AuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
```

```
            throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);

        String uri = request.getRequestURI();
        boolean loggedIn = session != null && session.getAttribute("loggedInAgent")
!= null;
        boolean isLoginPath = uri.endsWith("/login") || uri.endsWith("/login.jsp");

        if (loggedIn || isLoginPath) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

4. 日志/性能过滤器（前后包裹）：

```
@WebFilter(filterName = "LoggingFilter", urlPatterns = "/*")
public class LoggingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        long start = System.currentTimeMillis();
        System.out.println("→ " + request.getMethod() + " " +
request.getRequestURI());

        chain.doFilter(req, res); // 进入链路与目标资源

        long cost = System.currentTimeMillis() - start;
        System.out.println("← " + request.getRequestURI() + " cost=" + cost +
"ms");
    }
}
```

5. `web.xml` 配置与顺序（可选替代注解）：

```
<filter>
    <filter-name>EncodingFilter</filter-name>
    <filter-class>com.demo.web.filter.EncodingFilter</filter-class>
</filter>
<filter-mapping>
```

```
    <filter-name>EncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<filter>
    <filter-name>LoggingFilter</filter-name>
    <filter-class>com.demo.web.filter.LoggingFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>LoggingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>com.demo.web.filter.AuthFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/admin/*</url-pattern>
</filter-mapping>
```

- 注意：`web.xml` 中 `filter-mapping` 的先后顺序决定进入链的先后；退出顺序相反。
- 静态资源白名单：`/css/*`，`/js/*`，`/img/*` 等可跳过认证过滤。

教学方法

- 代码走查 + 示意图（洋葱模型）+ 小实验验证进入/退出顺序。

第三部分：实战——组装并验证请求处理链 (5分钟)

内容要点

1. 链路推荐顺序：`Logging`（最外层）→ `Encoding` → `Auth` → 业务Servlet/JSP。
2. 验证方法：在浏览器访问受保护与非受保护URL，观察控制台日志顺序与重定向行为。
3. 添加静态资源白名单，确认不被 `AuthFilter` 拦截。

教学方法

- 现场联调与对比测试（访问 `/admin/panel` vs `/public/home`）。

第四部分：课堂总结与展望 (2分钟)

总结要点

1. Filter是容器级“前后置”处理器，可形成可配置的处理链。
2. `chain.doFilter()` 前后即是“前置处理/后置处理”的钩子点。
3. 顺序与匹配策略决定链路行为，务必关注白名单与线程安全。

展望（承上启下）

- “守好入口”之后，还需要“感知全局生命周期”。下一讲将学习 `Listener` 监听器，完成应用启动初始化、在线人数统计与请求级追踪。

四、教学方法与手段

教学方法

1. **问题驱动 (Problem-Driven)**: 从重复代码痛点引出Filter
2. **示意图讲解 (Diagram)**: 洋葱模型展示Filter链
3. **代码走查 (Code Walkthrough)**: 逐步讲解Filter实现
4. **现场联调 (Live Testing)**: 验证Filter链执行顺序
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Filter, Filter Chain, Lifecycle, URL Pattern, Dispatcher Type）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述过滤器机制
    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 浏览器开发者工具 (Browser DevTools)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否正确解释过滤器链执行顺序与 `chain.doFilter()` 的前后时机；能否说出注解与 `web.xml` 配置的差异。
- **终结性评价**: 能否独立实现并验证编码过滤器、认证过滤器与日志过滤器的组合链路。

六、课后拓展

思考题

1. 如果在 `AuthFilter` 中省略对白名单资源的判断，会产生什么影响？如何在不改动代码的前提下通过 `web.xml` 降低影响面？
2. `DispatcherType.ERROR` 在什么场景下会触发过滤器？与 `REQUEST` 有何差别？

实践作业

1. 增加 `/static/*` 白名单：让 `AuthFilter` 跳过CSS/JS/图片资源。
2. 为 `LoggingFilter` 加入阈值告警：当请求耗时超过200ms时打印警告日志，并统计近N次平均耗时（可用 `AtomicLong` 与滑动窗口简单实现）。